Modeling a Large Data Set (Image Processing) with Singular Value Decomposition and Principal Component Analysis

Jimmy Sitompul*, Emma Dilley†, Maya Pachkowski‡, Kayla Robledo§

May 2023

Department of Mathematics, Colorado State University

Abstract

Through the use of Singular Value Decomposition and Principal Component Analysis, we performed several "transformations" on a data set containing images of dogs and cats to explore the process of facial recognition. For each image class, we found the average cat and dog images, as well as the symmetrized cat and dog images. After doubling the data set, we obtained clear eigencat and eigendog images. We obtained greater error when we projected a dog image onto the cat image class. We found a smaller dataset resulted in quick run time but our model would be improved with a larger data set that includes images in color with a variety of poses and lighting conditions.

1 Introduction

Have you ever wondered how facial recognition works and what the mathematics is behind it? This project focuses on just that. Facial recognition works by using eigenfaces. Eigenfaces are computed using Singular Value Decomposition (SVD) which is a concept in Linear Algebra. The development of SVD can be found in Section 2. Principal Component Analysis (PCA) is also used and the development can be found in Section 2.

In this project, we will be using images of cats and dogs instead of faces. Our data set contains 99 images of dogs and 99 images of cats which results in a total of 198 different images. Each image is 64 pixels $\times$ 64 pixels. Hence, each image contains 4096 pixels. The matrix has a total of 811008 ($64 \cdot 64 \cdot 198$) pixels where the first half (0 to 405504) are cats and the second half (405504

*Email: Jimmy.Sitompul@colostate.edu
†Email: Emma.Dilley@colostate.edu
‡Email: Maya.Pachkowski@colostate.edu
§Email: Kayla.Robledo@colostate.edu

to 811008) are dogs. These images are in grayscale. We will use this data to compute multiple different aspects of a photo. We will split the data into two classes, dogs and cats.

We will perform multiple "transformations" to each class. The following is what we do to each image: compute the average image for each class separately, take an image from one class and project it onto the other class, generate a basis, construct a projection matrix, mirror images and compute the eigenimage, and symmetrize the images using an even/odd basis. More on this can be found in Sections 2 and 3.

For this project we want to explore how well SVD and PCA work with cat and dog recognition. In specific, we investigate how well this works with grayscale images. We then perform image transformations- mirroring and symmetrizing and see if that will affect how well cat and dog recognition works.

We suspect that this model will do a decent job at recognizing cat and dog images. However, we suspect that mirroring and symmetrizing an image will not produce meaningful information.

The content of this paper is as follows: Section 2 describes the problem formulation, Section 3 describes the model development, Section 4 describes the results, Section 5 describes the robustness, Section 6 evaluates the model, and in Section 7 the extensions of the model are described. Finally, the conclusions of the report are addressed in Section 7.

2 Problem Formulation

2.1 Background of Singular Value Decomposition

Singular Value Decomposition, or SVD, is a classical dimension-reduction method that emphasizes the diagonalization of some data matrix into special matrices that we can analyze and manipulate. Specifically, we decompose our data into the composition of matrices:

$$U \cdot \sum \cdot V^T \tag{1}$$

where the columns of V and the rows of U are orthogonal.

$$U^T U = U U^T = I_{n \times n} \quad \text{and} \quad V^T V = V V^T = I_{n \times n} \tag{2}$$

and $\sum$ is a diagonal matrix with entries $\sigma_1, ..., \sigma_m$ such that

$$\sigma_1 \leq \sigma_2 \leq ... \leq \sigma_m \tag{3}$$

.

This gives the columns of U as the left singular vectors and the columns of V as the right singular vectors. Our data is a symmetric $n \times n$ matrix of 99 cats and 99 dogs. If we wanted to compute the SVD by hand, we would first find the characteristic polynomial of our matrix by calculating the determinant of

our data matrix, called A in this introductory section, but labeled as 'catdog' in our Python implementation, and its transpose subtracted by $\lambda \cdot I$. In other words,

$$AA^T - \lambda \cdot Iv \tag{4}$$

with A being our data matrix, wherein $\lambda \cdot I$ is an $n \times n$ diagonal matrix with $\lambda_1, ..., \lambda_n$ for diagonal entries. The determinant of any square matrix is defined as

$$\sum a_{ij} C_{ij} \tag{5}$$

where C_{ij} is the cofactor of a_{ij} defined as

$$C_{ij} \cong (-1)^{i+j} M_{ij} \tag{6}$$

and M_{ij} is the minor of A. The calculated determinant will be a polynomial of degree n with linear factors of the form $(\lambda - \lambda_i)$, with each λ_i denoted as the eigenvalues of AA^T and with each $\sigma_i = \sqrt{\lambda_i}$ denoted as the singular values of AA^T.

Using the relation between eigenvalues and singular values, we can form the matrix, $\sum$, noting that the singular values will be arranged in decreasing order along the diagonal. To find the matrix V, we need to find an orthonormal basis. We do this by determining and reducing to row echelon form $A^T A - \lambda_i \cdot I$. We call this new determination A' and solve the linear matrix equation $A_i^{'} \cdot v_i = 0$.

Finally, to determine the matrix U, it is easier to calculate $U = AV \sum^T$ and solve for U, as opposed to computing a basis using the eigenvalues of AA^T. The process of Singular Value Decomposition is then complete.

2.2 Relation to Principal Component Analysis

Principal Component Analysis, or PCA, is like SVD and is another classical dimension-reduction method that attempts to find linear combinations of features in some high dimensional data matrix to construct a meaningful representation of the data set. The resulting representations of each method are decidedly similar. In fact, SVD is often utilized in the process of PCA, especially for sparse matrices or frames of data.

Principal components can be determined by computing the eigen-decomposition of the covariance matrix C, given by:

$$C = \frac{A^T A}{n-1} \tag{7}$$

if our data is zero-centered. Now, the principal components of PCA can be determined by way of eigen-decomposing the covariance matrix C. However, here it is also possible to compute the SVD of C and yield the relationship between

the eigenvalues of the covariance matrix and the singular values. Observe:

$$\begin{aligned} C &= \frac{A^T A}{n-1} \\ &= V \sum U^T U \sum V^T \\ &= \frac{V \sum^2 V^T}{n-1} \\ &= \frac{V \sum^2 V^{-1}}{n-1} \end{aligned} \tag{8}$$

as V is unitary. Therefore, we yield that our eigenvalues are $\lambda_i = \frac{\sum_i^2}{n-1}$.

This relation implies that we can use either method, PCA or SVD, to reduce dimensionality! As mentioned earlier, many applications of PCA utilize SVD, or a truncated, an even more efficient variation of SVD not covered in this project, in their underlying processes.

2.3 Assumptions

There are no assumptions for Singular Value Decomposition. It is a procedure for finding a k-dimensional subspace and there are no prerequisites for doing so. Statisticians will often speak on distributions here, however, it is outside of the scope of this project. Principal Component Analysis additionally requires no assumptions. In the event that we conduct PCA on a sparse matrix, SVD is utilized on the covariance matrix.

2.4 Variable Definitions

Variables	**Definitions**
catdog	The matrix of the cat and dog images where the first 99 inputs are cats and the last 99 inputs are dogs
woof	Reshapes the dog picture into a column array rather than a row array. Both dog_pic and woof have the same number of elements and the same values
U	The left singular values, in a matrix, produced by SVD
S	The eigenvalues (a diagonal matrix) produced by SVD
eigen	Pulling off the eigenvalues from the U matrix
xmin	Gathering the minimum value of the eigenvalues which is used to convert the pixel values to grayscale images
xmax	Gathering the maximum value of the eigenvalues which is used to convert the pixel values to grayscale images
a	Used as the "slope" in the linear equations to convert the pixels to grayscale images
b	Used as the "intercept" in the linear equations to convert the pixels to grayscale images
f	The function used to convert our eigenvalues to the grayscale image
eigencats	The eigencat image is a 64 by 64 matrix of image type (uint8). This is used to produce the eigencat image
projection_dog	The matrix multiplication of U, U^T, and woof, where UU^T is $\mathbb{P}$
error_dog	Takes the variable woof and subtracts dog_projection
meow	Reshapes the cat picture into a column array rather than a row array. Both cat_pic and woof have the same number of elements and the same values
eigendogs	The eigendog image that is a 64 by 64 matrix of image type (uint8). This is used to produce the eigendog image
projection_cat	The matrix multiplication of U, U^T, and meow, where UU^T is $\mathbb{P}$
error_cat	Takes the variable meow and subtracts cat_projection
kitty_cat	The transposed matrix of cat and cat mirror combined. These are reshaped into a column array
doggy_dog	The transposed matrix of dog and dog mirror combined. These are reshaped into a column array
sym _dog	The symmetrized dog image by adding the even component to itself reflected about the vertical axis and subtracting the odd component from itself reflected about the vertical axis
sym_cat	The symmetrized cat image by adding the even component to itself reflected about the vertical axis and subtracting the odd component from itself reflected about the vertical axis
symmetrized_dog	The sym_dog image that is a 64 by 64 matrix of image type (uint8)
symmetrized_cat	The sym_cat image that is a 64 by 64 matrix of image type (uint8)

3 Model Development

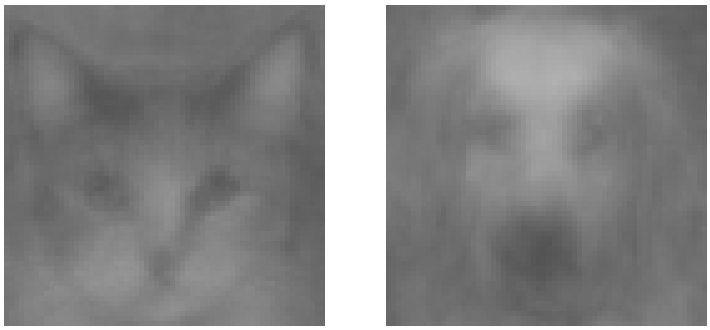
We are employing a data set of 99 images of dogs and 99 images of cats, wherein each image is 64×64 pixels. Using Python built-in packages of Numpy, Scipy, Pandas, and Sklearn Preprocessing, we will compute the average image, take a single image and project it onto the species class, and finally display the resulting eigencat and/or eigendog across both species, respectively. We make use specifically of Numpy's built-in SVD function to extract the left singular values of matrix U and subsequently display the principal components derived from the columns of U. Additionally, our program utilizes a Python module, cv2, designed for visualization and displaying images. All the code for this can be found in Section 9.

To compute the average image across cats and dogs, we extract the photos of cats separately from the images of the dogs and flatten the images into arrays of a data type that will allow us to call the Numpy function mean. We then take an image x for each class C_i for $i = cat$ or dog, corresponding to species, and project the image onto the class of the other species to see the error of the image with respect to the wrong class. In other words, we take an image of some cat, project it onto the class of images of dogs, and note the significant inaccuracies of the resulting image. By holding out a single class and repeating the computation of an average image for the held-out class, we will assess how much the image subspace represents the class. We will finish the process by determining our left singular values, doubling the size of our data by including mirror images of each image, and determining the resulting eigenfaces (by taking the columns of U) and symmetrized images via the basis of even and odd vectors using PCA.

We have various small programs, shown in Section 9, wherein we create and call the configured methods and functions for the above process. Our goal is to effectively show that Singular Value Decomposition, as well as Principal Component Analysis, truly works to gather the most useful representation of data and thus, it is fundamental to facial recognition in machine learning.

4 Results

4.1 Average Cats and Dogs



(a) Average Cat Image (b) Average Dog Image

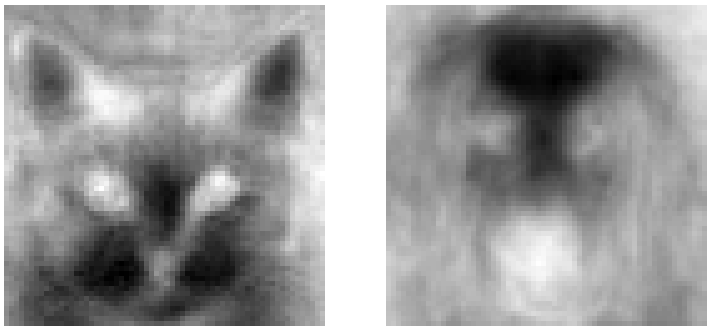
Figure 1: The Average Face of **(a)** 99 Cat Images and **(b)** 99 Dog Images

First, we calculated the average cat and dog image from a matrix contained in the **catdog.mat** file. The matrix is initially separated into cat and dog matrices. Each matrix is then transformed into an array of pixels, and the average value of each pixel is calculated. The resulting arrays are then converted back into images representing the average cat and dog images shown in **Figure 1**.

4.2 Eigencats and Eigendogs

We used the SVD formula in equation **(1)** for each cat and dog matrix to find a sample eigenface for cats (or eigencat) and a sample eigenface for dogs (or eigendog). After computing the SVD for each cat and dog matrix, we obtained the left singular matrix U and we noticed that U_k denotes the k-th eigenface of each cat and dog image.

Here, we picked the first column of the left singular matrix U, denoted by U_0, to compute the first eigencat and eigendog. Since the norm $\|U_k\| = 1$, we have x_i to be the entries of U_k such that $x_i \in (-1, 1)$ for all $x_i \in \mathbb{R}$ and $i = 0, \ldots, \text{len}(U_k) - 1$. In other words, x_i's are decimal values ranging from -1 to 1 exclusive, so these have to be transformed into integers from 0 to 255, denoting the pixel values of a grayscale image.



(a) Eigencat Image (b) Eigendog Image

Figure 2: The Images of The Resulting Eigenface of Cats and Dogs

In order to do so, we set up the equations:

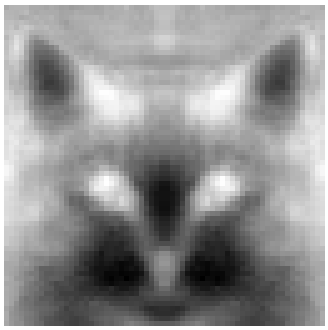
$$\begin{aligned} f(x_{\min}) &= a \cdot x_{\min} + b = 0 \\ f(x_{\max}) &= a \cdot x_{\max} + b = 255 \end{aligned} \tag{9}$$

$$\implies \begin{pmatrix} x_{\min} & 1 \\ x_{\max} & 1 \end{pmatrix} \begin{pmatrix} a \\ b \end{pmatrix} = \begin{pmatrix} 0 \\ 255 \end{pmatrix}$$

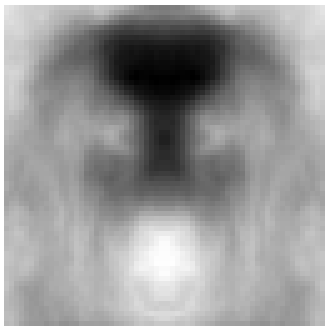
Solving for a and b, we obtained the mapping from the initial x_i values to the pixel values $f(x_i)$ is defined by:

$$f(x_i) = ax_i + b. \tag{10}$$

After transforming into an array of pixel values, the array is converted into images representing an eigencat and an eigendog, or representing U_0. According to **Figure 2 (a)**, we have the first eigencat efficiently represents the collection of cat images, whereas the eigendog has a blurry image and it does not perform as well as eigencat does in image recognition. When we double each cat and dog data set by taking the mirror of every image in the cat and dog data set, we will have more accurate results shown in **Figure 3** as we have a more symmetrized and clear images of eigencat and eigendog in this case.



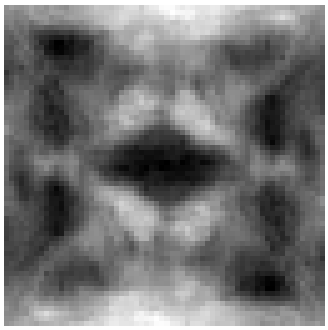
(a) Eigencat Image (Mirrored and Doubled-size)



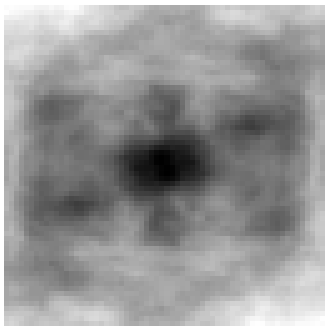
(b) Eigendog Image (Mirrored and Doubled-size)

Figure 3: The Images of The Resulting Eigenface of The Mirrored and Doubled-size Cats and Dogs

We also considered having the rotated version of the cat and dog images, by taking the mirror of every image, and computing the eigencat and eigendog images of the rotated version of the cat and dog data set. As shown in **Figure 4**, we have the eigendog and the eigencat images that strangely represent the cat and dog images in the data set.



(a) Eigencat Image (Rotated, Mirrored, and Doubled-size)



(b) Eigendog Image (Rotated, Mirrored, and Doubled-size)

Figure 4: The Images of The Resulting Eigenface of The Rotated, Mirrored, and Doubled-size Cats and Dogs

Here, we have the eigencat and the eigendog images represent the horizontally-mirrored of the upper half images of the cats and dogs. Although the resulting eigendog and eigencat images in **Figure 4** are very unusual compared to the results shown in **Figure 2** and **Figure 3**, these results will be beneficial for image recognition as in involving a collection of images under different conditions (i.e. different orientation in this case).

4.3 Image Projection and Error

By projecting a cat image to the set of dog images class, SVD allows us to classify the cat image as a dog image or not based on its similarity to the dog images in the data set. By projecting a cat image onto the space of dog images using SVD, we can extract the most important features that characterize the dog images and determine how closely the cat image aligns with those features.

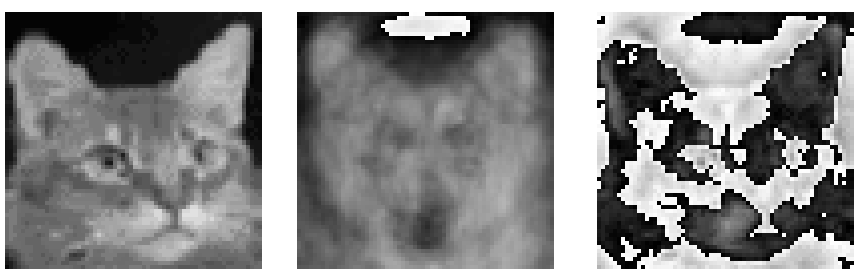
Here, we took a cat image from the cat class, and we then projected the cat image onto the dog class by computing the SVD for the dog class. We selected the first k columns of the left singular matrix U of the dog class and took the transpose of the first k columns of U. We used the following equations to display (11) the projected cat image x and (12) the projection error image onto the dog class:

$$\begin{aligned} \mathbb{P}x &= (U_k U_k^T)x \\ &= U_k(U_k^T x) \end{aligned} \tag{11}$$

and

$$\begin{aligned} (I - \mathbb{P})x &= x - \mathbb{P}x \\ &= x - (U_k U_k^T)x \end{aligned} \tag{12}$$

After calculating the projected image matrix $\mathbb{P}x$ and the projection error image matrix $(I - \mathbb{P})x$, we convert it back into images, as shown in **Figure 5** and **Figure 6**.



(a) Cat Image (b) Projected Cat Image (c) Error Cat Image

Figure 5: The Projection of a Cat Image to the Dog Class and Its Projection Error

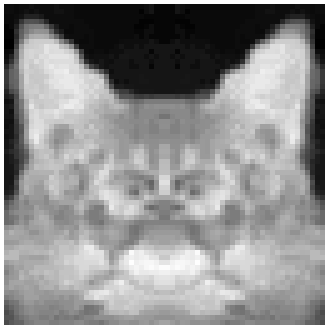
(a) Dog Image (b) Projected Dog Image (c) Error Dog Image

Figure 6: The Projection of a Dog Image to the Cat Class and Its Projection Error

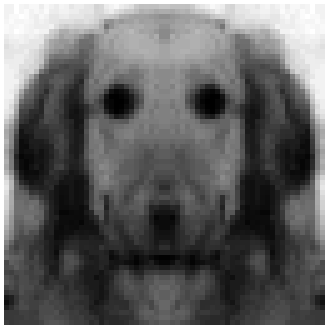
In **Figure 5**, the projected cat image looks quite similar to the dog image class in terms of feature and appearance, thus, there is a possibility that the cat image shares some of the same characteristics as dog images. However, we noticed that the projected cat image is not even that close to the dog image since the projected image has cat ears despite having a dog-looking face. On the other hand, the projected dog image, shown in **Figure 6**, looks very similar to the cat image class as the projected dog image shares almost the same characteristics as dog images.

This can be seen from how much error is displayed when we project a cat image onto the dog image class and vice versa. Here, the error displayed when we projected a dog image onto the cat image class is more significant compared to when we projected a cat image onto the dog image class. Therefore, the projected dog image is significantly different from the original dog image and may be classified as a cat image since it looks very similar to the cat image class. Similarly, we can say that the projected cat image also can be classified as a dog image, but the projection of the cat image is not as good as the projection of the dog image.

4.4 Symmetrized Cat and Dog Images



(a) Eigencat Image (Mirrored and Doubled-size)



(b) Eigendog Image (Mirrored and Doubled-size)

Figure 7: The Images of The Resulting Eigenface of The Mirrored and Doubled-size Cats and Dogs

In **Figure 7**, we have the symmetrized image of the cat and the dog, which can be obtained by constructing even and odd basis vectors. In this method, the cat and dog images are decomposed into their even and odd components, with the even component being the symmetric part of the image and the odd component being the anti-symmetric part of the image. The even and odd components can be reconstructed separately, and then combined to create a symmetrized version of the original image.

5 Robustness

This model was strong for the given data set. The code only took about 2 minutes maximum to run everything. It was able to compute all of the data given in Section 4. It should be noted that the largest the data set got was double its original size. This would be 198 images in each data set because we split the data into two (99 images in each data set).

Due to this small data set, we were able to use the SVD function in Python to compute our left singular values and eigenvalues. However, if the data set was larger, this code would take too long to run and we would have to approach this differently. Thus, if this code was applied to an extremely large data set, the model would not be very robust.

Additionally, rotating each image added to the robustness of this model. This is because rotating the image allows us to get a better understanding of a cat's or dog's facial features. Thus, we can see what details of each image are crucial for facial recognition.

Reasons, why the model was not robust, will be mentioned in greater detail in the latter section, Section 6.3. A few mentions are that we did not

know the conditions of each of the photos. Thus, it is not clear if each animal posed differently or if the photo was taken in different lighting conditions. This affected the model's robustness because it is not clear if there were 99 different cats or the same cat with 99 different poses and/or lighting conditions and the same thing for dogs.

There is still great value that was learned from this model. Although the model would not be robust for massive data for real-world applications, it is a strong model for small data sets.

6 Model Evaluation

6.1 Added Value

After analyzing both the eigencat and eigendog images, we noticed that if we doubled the data set by adding mirrored cat and dog that have similar orientations to the existing data set, the eigendog and eigencat may not change significantly. However, adding more variety of cat and dog images (i.e. different poses, orientations, and lighting) can help improve face recognition accuracy. It is important to note that the quality of the data set and the selection of cat and dog images are crucial in determining the effectiveness of eigencat- and eigendog-based face recognition.

6.2 Model Strengths

Using SVD in our model is computationally time-efficient and can be performed quickly on large data sets. This makes it a practical solution for real-time applications such as face recognition systems. Furthermore, we included the eigencat and the eigendog of the rotated and mirrored images. In our model, we also have an eigencat and eigendog resulting from different image rotations to provide a more robust representation of facial features.

Another strength of using SVD in our model is that it can enhance the interpretability of the results. SVD decomposes the original image into three matrices: U, Σ, and V. These matrices contain information about the image's singular vectors, singular values, and transposed singular vectors. By manipulating these matrices, it is possible to project the original image onto a different class or generate a new one altogether. This can provide insights into the underlying structure of the image and facilitate the identification of relevant patterns or features.

6.3 Model Weaknesses

Our model could be improved upon by using the colored cat and dog images data set instead of the grayscale cat and dog images data set. Color information is important for image processing, but since grayscale images lack color information, this may limit the accuracy of the displayed images from the code.

Additionally, using a grayscale data set makes the displayed images difficult to be seen and analyzed.

Another weakness of this model is that we have a small amount of cat and dog data (99 images each) which may not provide enough information for the code to display accurate images and identify all possible poses of the cat and dog images being analyzed. Furthermore, limited poses may not provide enough variability for the code to accurately identify and represent the object in different orientations and positions, which can also limit the accuracy of the images obtained from the code.

7 Extensions of the Model

- To build off of this specific model, there are several variations that can be done. Namely, we could begin by addressing the weaknesses in our model. For example, we could expand upon the project by using colored images rather than grayscale images. We could use classes that include larger sets of images. We could use classes with more diverse images in terms of lighting, positions, etcetera.

- We chose to do this project with dogs and cats rather than the original intensive purpose which was human faces. Thus, an obvious extension of this model is to repeat this process with human faces, rather than animals. Should this project have been done with human images, the model could have been extended for several purposes. The process of SVD is often associated with the computation of 'eigenfaces' which are used in facial recognition software [2].

- Another extension of our model is to implement sorting and grouping algorithms to distinguish between classes by means of machine learning. This is not the first time this type of model has been used in this way: "Eigenfaces is a method that is useful for face recognition and detection by determining the variance of faces in a collection of face images and use those variances to encode and decode a face in a machine learning way without the full information reducing computation and space complexity" [1].

sectionConclusions Through the use of Singular Value Decomposition and Principal Component Analysis, we performed several "transformations" on a data set containing 99 images of dogs and 99 images of cats in order to explore the process of facial recognition. Each image was 64×64 pixels and each animal type made up its own image class. For each image class, we successfully found the average cat and dog images, as well as the symmetrized cat and dog images. After including the mirror images of the data set, we were able to obtain more clear eigencat and eigendog images. When projecting images from one class onto the other, we obtain greater error when we projected a dog image onto the cat image class.

With our small dataset, we saw success in run time and our ability to use the SVD function in Python, though the ambiguity of our dataset was cause for concern. That being said, by doubling the dataset, we were able to improve accuracy by increasing variability. We would benefit from further exploration with different datasets that include images in color, larger data sets, and data sets that include images with a variety of poses and lighting conditions. This model could be extended for purposes of facial recognition of human images in the realm of machine learning.

8 Bibliography

[1] N. Acar. *Eigenfaces: Recovering Humans from Ghosts*. 2018. URL: https://towardsdatascience.com/eigenfaces-recovering-humans-from-ghosts-17606c328184.

[2] N. Muller, L. Magaia, and B.M. Herbst. *Singular Value Decomposition, Eigenfaces, and 3D Reconstructions*. 2004. URL: https://epubs.siam.org/doi/pdf/10.1137/S0036144501387517.

9 Code

9.1 Average Cats and Dogs

```
import numpy as np
import cv2  # OpenCV library for image processing
from scipy.io import loadmat  # Library for reading MATLAB files
import pandas as pd  # Library for data manipulation and analysis


# Load the MATLAB file containing cat and dog images
mat = loadmat('C:/Users/Jimmy Sitompul/Downloads/catdog.mat')

# Print the contents of the MATLAB file
print(mat)

# Extract the data from the MATLAB file and convert it to
# a Pandas DataFrame
mat1 = pd.DataFrame(mat.get('T'))

# Convert the data to a NumPy array of float32 data type and flatten it
mat2 = np.array(mat1, dtype=np.float32).ravel()

# Print the length of the flattened NumPy array
print(len(mat2))

# Extract the first 99 images (representing cats) from the flattened array
# and reshape it to a 2D array of length 99 whose elements are sub-arrays
# of length 4096
```

```python
cat = np.array(mat2[0:405504], dtype=np.float32)
cat = np.reshape(cat, (99, 4096))
for i in range (0,99):
    img = cat[i]
    img = np.reshape(img, (64, 64))
    img = np.rot90(img, k=3)
    img = np.array(img, dtype=np.float32).ravel()
    cat[i]=img

# Extract the next 99 images (representing dogs) from the flattened array
# and reshape it to a 2D array of length 99 whose elements are sub-arrays
# of length 4096
dog = np.array(mat2[405504:811008], dtype=np.float32)
dog = np.reshape(dog, (99, 4096))
for i in range (0,99):
    img = dog[i]
    img = np.reshape(img, (64, 64))
    img = np.rot90(img, k=3)
    img = np.array(img, dtype=np.float32).ravel()
    dog[i]=img

# Print the cat and dog arrays and their lengths
print(len(cat))
print(len(dog))

# Calculate the average of the cat and dog arrays along
# the columns (i.e., across the images)
avg_cat = np.mean(cat, axis=0)
print(avg_cat)
avg_dog = np.mean(dog, axis=0)
print(avg_dog)

# Reshape the average cat and dog arrays to 2D arrays of 64x64 size
# and convert them to unsigned integer data type
avg_image_cat = np.reshape(avg_cat, (64, 64)).astype(np.uint8)
avg_image_dog = np.reshape(avg_dog, (64, 64)).astype(np.uint8)

# Display the average cat and dog images using OpenCV's imshow function
cv2.imshow("Cat Average Face", avg_image_cat)
cv2.imshow("Dog Average Face", avg_image_dog)

# Wait for a key press and destroy all windows
cv2.waitKey(0)
cv2.destroyAllWindows()
```

9.2 Dog Projection and Error Image

```python
import numpy as np
import cv2  # OpenCV library for image processing
from scipy.io import loadmat  # Library for reading MATLAB files
from scipy.linalg import svd
import pandas as pd  # Library for data manipulation and analysis

```

```python
# Load the MATLAB file containing cat and dog images
mat = loadmat('C:/Users/Jimmy Sitompul/Downloads/catdog.mat')

# Print the contents of the MATLAB file
print(mat)

# Extract the data from the MATLAB file and convert it to
# a Pandas DataFrame
mat1 = pd.DataFrame(mat.get('T'))

# Convert the data to a NumPy array of float32 data type and flatten it
mat2 = np.array(mat1, dtype=np.float32).ravel()

# Print the length of the flattened NumPy array
print(len(mat2))

# Extract the first 99 images (representing cats) from the flattened array
# and reshape it to a 2D array of length 99 whose elements are sub-arrays
# of length 4096
cat = np.array(mat2[0:405504], dtype=np.float32)
cat = np.reshape(cat, (99, -1))
for i in range (0,99):
    img = cat[i]
    img = np.reshape(img, (64, 64))
    img = np.rot90(img, k=3)
    img = np.array(img, dtype=np.float32).ravel()
    cat[i]=img
cat = cat.T

# Extract the next 99 images (representing dogs) from the flattened array
# and reshape it to a 2D array of length 99 whose elements are sub-arrays
# of length 4096
dog = np.array(mat2[405504:811008], dtype=np.float32)
dog = np.reshape(dog, (99, -1))
for i in range (0,99):
    img = dog[i]
    img = np.reshape(img, (64, 64))
    img = np.rot90(img, k=3)
    img = np.array(img, dtype=np.float32).ravel()
    dog[i]=img
dog_pic = dog[0]

# Print the cat and dog arrays and their lengths
print('Dog Pic Array')
print(dog_pic)
print('Cat Array')
print(cat)

woof = np.reshape(dog_pic,(len(dog_pic),-1))

U,S,V_T = svd(cat, full_matrices=False)

```

```
print('Left Singular')
print(U)
print('Eigenvalues (diagonal)')
print(S)

k = 10

eigen=U[:,0]
print('Eigen')
print(eigen)

xmin= min(eigen)
xmax = max(eigen)

print('X minimum')
print(xmin)
print('X maximum')
print(xmax)

eigencats=np.array([])

for i in eigen:
    f= lambda i: 33109.26153*i+651.84401
    val = int(f(i))
    eigencats = np.append(eigencats,val)

print('Eigencats')
print(eigencats)

eigencats = np.reshape(eigencats,(64,64)).astype(np.uint8)
cv2.imshow('Eigencat Image', eigencats)

projection_dog = np.dot(U[:,0:k],np.dot(U[:,0:k].T,woof))
error_dog = woof - projection_dog

woof_pic = np.reshape(woof, (64, 64)).astype(np.uint8)
projection_dog_pic = np.reshape(projection_dog, (64, 64)).astype(np
    .uint8)
error_dog_pic = np.reshape(error_dog, (64, 64)).astype(np.uint8)

cv2.imshow('Dog Image', woof_pic)
cv2.imshow('Projected Dog Image', projection_dog_pic)
cv2.imshow('Error Dog Image', error_dog_pic)

cv2.waitKey(0)
cv2.destroyAllWindows()
```

9.3 Cat Projection and Error Image

```
import numpy as np
import cv2  # OpenCV library for image processing
from scipy.io import loadmat  # Library for reading MATLAB files
from scipy.linalg import svd
import pandas as pd  # Library for data manipulation and analysis

# Load the MATLAB file containing cat and dog images
```

```python
mat = loadmat('C:/Users/Jimmy Sitompul/Downloads/catdog.mat')

# Print the contents of the MATLAB file
print("Matlab File")
print(mat)

# Extract the data from the MATLAB file and convert it to
# a Pandas DataFrame
mat1 = pd.DataFrame(mat.get('T'))

# Convert the data to a NumPy array of float32 data type and flatten it
mat2 = np.array(mat1, dtype=np.float32).ravel()

# Print the length of the flattened NumPy array
print("Length of Array")
print(len(mat2))

# Extract the first 99 images (representing cats) from the flattened array
# and reshape it to a 2D array of length 99 whose elements are sub-arrays
# of length 4096
cat = np.array(mat2[0:405504], dtype=np.float32)
cat = np.reshape(cat, (99, -1))
for i in range (0,99):
    img = cat[i]
    img = np.reshape(img, (64, 64))
    img = np.rot90(img, k=3)
    img = np.array(img, dtype=np.float32).ravel()
    cat[i]=img

cat_pic = cat[0]

# Extract the next 99 images (representing dogs) from the flattened array
# and reshape it to a 2D array of length 99 whose elements are sub-arrays
# of length 4096
dog = np.array(mat2[405504:811008], dtype=np.float32)
dog = np.reshape(dog, (99, -1))
for i in range (0,99):
    img = dog[i]
    img = np.reshape(img, (64, 64))
    img = np.rot90(img, k=3)
    img = np.array(img, dtype=np.float32).ravel()
    dog[i]=img

dog = dog.T

# Print the cat and dog arrays and their lengths
print('Cat Pic Array')
print(cat_pic)
print('Dog Array')
print(dog)

meow = np.reshape(cat_pic,(len(cat_pic),-1))
```

```
print(meow.shape)

U,S,V_T = svd(dog, full_matrices=False)

print('Left Singular')
print(U)
print('Eigenvalues (diagonal)')
print(S)

k = 10
print(U[:,0:k].shape)

eigen = U[:,0]
eigen = np.array(eigen,dtype=np.float32).ravel()
print(eigen)

xmin= min(eigen)
xmax = max(eigen)

print('X minimum')
print(xmin)
print('X maximum')
print(xmax)

eigendogs=np.array([])

for i in eigen:
    f = lambda i : 21713.37796*i+480.94155
    val = int(f(i))
    eigendogs = np.append(eigendogs,val)

print("Eigendogs")
print(eigendogs)

eigendogs = np.reshape(eigendogs,(64,64)).astype(np.uint8)
cv2.imshow('Eigendog Image', eigendogs)

projection_cat = np.dot(U[:,0:k],np.dot(U[:,0:k].T,meow))
error_cat = meow - projection_cat

meow_pic = np.reshape(meow, (64, 64)).astype(np.uint8)
projection_cat_pic = np.reshape(projection_cat, (64, 64)).astype(np
    .uint8)
error_cat_pic = np.reshape(error_cat, (64, 64)).astype(np.uint8)

cv2.imshow('Cat Image', meow_pic)
cv2.imshow('Projected Cat Image', projection_cat_pic)
cv2.imshow('Error Cat Image', error_cat_pic)

cv2.waitKey(0)
cv2.destroyAllWindows()
```

9.4 Eigencat

```
import numpy as np
import cv2  # OpenCV library for image processing
```

```python
from scipy.io import loadmat  # Library for reading MATLAB files
from scipy.linalg import svd
import pandas as pd  # Library for data manipulation and analysis
import numpy as np
from sklearn.preprocessing import normalize

# Load the MATLAB file containing cat and dog images
mat = loadmat('C:/Users/Jimmy Sitompul/Downloads/catdog.mat')

# Print the contents of the MATLAB file
print(mat)
print(len(mat.get("T")))
print([len(a) for a in mat.get("T")])

# Extract the data from the MATLAB file and convert it to
# a Pandas DataFrame
mat1 = pd.DataFrame(mat.get("T"))

# Convert the data to a NumPy array of float32 data type and flatten it
mat2 = np.array(mat1, dtype=np.float32).ravel()

# Print the length of the flattened NumPy array
print(len(mat2))
print(len(mat2)/2)

# Extract the first 99 images (representing cats) from the flattened array
# and reshape it to a 2D array of length 99 whose elements are sub-arrays
# of length 4096
cat = np.array(mat2[0:405504], dtype=np.float32) #99*4096 = 405504
cat = np.reshape(cat, (99, -1))


for i in range (0,99):
    img = cat[i]
    img = np.reshape(img, (64, 64))
    img = np.rot90(img, k=3)
    img = np.array(img, dtype=np.float32).ravel()
    cat[i]=img


cat = np.reshape(cat, (198, -1))
cat = cat.T
print(cat.shape)

# Extract the next 99 images (representing dogs) from the flattened array
# and reshape it to a 2D array of length 99 whose elements are sub-arrays
# of length 4096
dog = np.array(mat2[405504:811008], dtype=np.float32) #405504*2 = 811008
dog = np.reshape(dog, (99, -1))
for i in range (0,99):
    img = dog[i]
```

```
    img = np.reshape(img, (64, 64))
    img = np.rot90(img, k=3)
    img = np.array(img, dtype=np.float32).ravel()
    dog[i]=img

# Print the cat and dog arrays and their lengths


U,S,V_T = svd(cat, full_matrices=False)

k = 10


eigen = U[:,0]
eigen = np.array(eigen,dtype=np.float32).ravel()
print(len(eigen))

xmin= min(eigen)
xmax = max(eigen)

print('X minimum')
print(xmin)
print('X maximum')
print(xmax)

eigencats=np.array([])

for i in eigen:
    f = lambda i : 35722.34075*i+697.96309
    val = int(f(i))
    eigencats = np.append(eigencats,val)

print("Eigencats")
print(eigencats)

eigencats = np.reshape(eigencats,(64,64)).astype(np.uint8)
cv2.imshow('Eigencat Image', eigencats)

cv2.waitKey(0)
cv2.destroyAllWindows()
```

9.5 Eigencat - Rotated

```
import numpy as np
import cv2  # OpenCV library for image processing
from scipy.io import loadmat  # Library for reading MATLAB files
from scipy.linalg import svd
import pandas as pd  # Library for data manipulation and analysis
import numpy as np
from sklearn.preprocessing import normalize

# Load the MATLAB file containing cat and dog images
mat = loadmat('C:/Users/Jimmy Sitompul/Downloads/catdog.mat')

# Print the contents of the MATLAB file
print(mat)
```

```python
print(len(mat.get("T")))
print([len(a) for a in mat.get("T")])

# Extract the data from the MATLAB file and convert it to
# a Pandas DataFrame
mat1 = pd.DataFrame(mat.get('T'))

# Convert the data to a NumPy array of float32 data type and flatten it
mat2 = np.array(mat1, dtype=np.float32).ravel()

# Print the length of the flattened NumPy array
print(len(mat2))
print(len(mat2)/2)

# Extract the first 99 images (representing cats) from the flattened array
# and reshape it to a 2D array of length 99 whose elements are sub-arrays
# of length 4096
cat = np.array(mat2[0:405504], dtype=np.float32) #99*4096 = 405504
cat = np.reshape(cat, (99, -1))
cat_mirror = np.fliplr(cat)
cat = np.array(cat, dtype=np.float32)
cat_mirror = np.array(cat_mirror, dtype=np.float32)

kitty_cat = np.append(cat,cat_mirror)
kitty_cat = np.reshape(kitty_cat, (198,-1))
kitty_cat = kitty_cat.T

# Extract the next 99 images (representing dogs) from the flattened array
# and reshape it to a 2D array of length 99 whose elements are sub-arrays
# of length 4096
dog = np.array(mat2[405504:811008], dtype=np.float32) #405504*2 = 811008
dog = np.reshape(dog, (99, -1))
dog_pic = dog[0]

dog_mirror = np.fliplr(dog)
dog = np.array(dog, dtype=np.float32)
dog_mirror = np.array(dog_mirror, dtype=np.float32)

doggy_dog = np.append(dog,dog_mirror)
doggy_dog = np.reshape(doggy_dog,(198,-1))
doggy_dog = doggy_dog.T

# Print the cat and dog arrays and their lengths
print(dog_pic)
print(cat)

woof = np.reshape(dog_pic,(len(dog_pic),-1))

U,S,V_T = svd(kitty_cat, full_matrices=False)

k = 10
```

```


eigen = U[:,0]
eigen = np.array(eigen,dtype=np.float32).ravel()
print(eigen)

xmin= min(eigen)
xmax = max(eigen)

print('X minimum')
print(xmin)
print('X maximum')
print(xmax)

eigendogs=np.array([])

for i in eigen:
    f = lambda i : 51918.40544*i+919.15093
    val = int(f(i))
    eigendogs = np.append(eigendogs,val)

print("Eigendogs")
print(eigendogs)

eigendogs = np.reshape(eigendogs,(64,64)).astype(np.uint8)
cv2.imshow('Eigendog Image', eigendogs)

cv2.waitKey(0)
cv2.destroyAllWindows()
```

9.6 Eigendog

```
import numpy as np
import cv2  # OpenCV library for image processing
from scipy.io import loadmat  # Library for reading MATLAB files
from scipy.linalg import svd
import pandas as pd  # Library for data manipulation and analysis
import numpy as np
from sklearn.preprocessing import normalize

# Load the MATLAB file containing cat and dog images
mat = loadmat('C:/Users/Jimmy Sitompul/Downloads/catdog.mat')

# Print the contents of the MATLAB file
print(mat)
print(len(mat.get("T")))
print([len(a) for a in mat.get("T")])

# Extract the data from the MATLAB file and convert it to
# a Pandas DataFrame
mat1 = pd.DataFrame(mat.get('T'))

# Convert the data to a NumPy array of float32 data type and
    flatten it
mat2 = np.array(mat1, dtype=np.float32).ravel()

```

```
# Print the length of the flattened NumPy array
print(len(mat2))
print(len(mat2)/2)

# Extract the first 99 images (representing cats) from the flattened array
# and reshape it to a 2D array of length 99 whose elements are sub-arrays
# of length 4096
cat = np.array(mat2[0:405504], dtype=np.float32) #99*4096 = 405504
cat = np.reshape(cat, (99, -1))
for i in range (0,99):
    img = cat[i]
    img = np.reshape(img, (64, 64))
    img = np.rot90(img, k=3)
    img = np.array(img, dtype=np.float32).ravel()
    cat[i]=img



# Extract the next 99 images (representing dogs) from the flattened array
# and reshape it to a 2D array of length 99 whose elements are sub-arrays
# of length 4096
dog = np.array(mat2[405504:811008], dtype=np.float32) #405504*2 = 811008
dog = np.reshape(dog, (99, -1))
for i in range (0,99):
    img = dog[i]
    img = np.reshape(img, (64, 64))
    img = np.rot90(img, k=3)
    img = np.array(img, dtype=np.float32).ravel()
    dog[i]=img

for img in dog:
    img = np.reshape(img, (64, 64))
    mirror_img = np.fliplr(img)
    mirror_img = np.array(mirror_img, dtype=np.float32).ravel()
    dog = np.append(dog,mirror_img)

dog = np.reshape(dog, (198, -1))
dog = dog.T
print(dog.shape)

U,S,V_T = svd(dog, full_matrices=False)

k = 10

eigen = U[:,0]
eigen = np.array(eigen,dtype=np.float32).ravel()
print(eigen)

xmin= min(eigen)
xmax = max(eigen)

print('X minimum')
```

```
print(xmin)
print('X maximum')
print(xmax)

eigendogs=np.array([])

for i in eigen:
    f = lambda i : 23017.78299*i+504.26438
    val = int(f(i))
    eigendogs = np.append(eigendogs,val)

print("Eigendogs")
print(eigendogs)

eigendogs = np.reshape(eigendogs,(64,64)).astype(np.uint8)
cv2.imshow('Eigendog Image', eigendogs)

cv2.waitKey(0)
cv2.destroyAllWindows()
```

9.7 Eigendog - Rotated

```
import numpy as np
import cv2  # OpenCV library for image processing
from scipy.io import loadmat  # Library for reading MATLAB files
from scipy.linalg import svd
import pandas as pd  # Library for data manipulation and analysis
import numpy as np
from sklearn.preprocessing import normalize

# Load the MATLAB file containing cat and dog images
mat = loadmat('C:/Users/Jimmy Sitompul/Downloads/catdog.mat')

# Print the contents of the MATLAB file
print(mat)
print(len(mat.get("T")))
print([len(a) for a in mat.get("T")])

# Extract the data from the MATLAB file and convert it to
# a Pandas DataFrame
mat1 = pd.DataFrame(mat.get('T'))

# Convert the data to a NumPy array of float32 data type and
    flatten it
mat2 = np.array(mat1, dtype=np.float32).ravel()

# Print the length of the flattened NumPy array
print(len(mat2))
print(len(mat2)/2)

# Extract the first 99 images (representing cats) from the
    flattened array
# and reshape it to a 2D array of length 99 whose elements are sub-
    arrays
# of length 4096
cat = np.array(mat2[0:405504], dtype=np.float32) #99*4096 = 405504
```

```
cat = np.reshape(cat, (99, -1))
cat_mirror = np.fliplr(cat)
cat_pic = cat[0]

# Extract the next 99 images (representing dogs) from the flattened array
# and reshape it to a 2D array of length 99 whose elements are sub-arrays
# of length 4096
dog = np.array(mat2[405504:811008], dtype=np.float32) #405504*2 = 811008
dog = np.reshape(dog, (99, -1))


dog_mirror = np.fliplr(dog)
dog = np.array(dog, dtype=np.float32)
dog_mirror = np.array(dog_mirror, dtype=np.float32)

doggy_dog = np.append(dog,dog_mirror)
doggy_dog = np.reshape(doggy_dog,(198,-1))
doggy_dog = doggy_dog.T

# Print the cat and dog arrays and their lengths
print(cat_pic)
print(dog)

meow = np.reshape(cat_pic,(len(cat_pic),-1))

U,S,V_T = svd(doggy_dog, full_matrices=False)

k = 10


eigen = U[:,0]
eigen = np.array(eigen,dtype=np.float32).ravel()
print(eigen)

xmin= min(eigen)
xmax = max(eigen)

print('X minimum')
print(xmin)
print('X maximum')
print(xmax)

eigencats=np.array([])

for i in eigen:
    f = lambda i : 33170.57421*i+663.41868
    val = int(f(i))
    eigencats = np.append(eigencats,val)

print("Eigencats")
print(eigencats)

eigencats = np.reshape(eigencats,(64,64)).astype(np.uint8)
cv2.imshow('Eigencat Image', eigencats)
```

```

cv2.waitKey(0)
cv2.destroyAllWindows()
```

9.8 Symmetrized Cat Image

```
import numpy as np
import cv2  # OpenCV library for image processing
from scipy.io import loadmat  # Library for reading MATLAB files
from scipy.linalg import svd
import pandas as pd  # Library for data manipulation and analysis
import numpy as np
from sklearn.preprocessing import normalize
from sklearn.decomposition import PCA

# Load the MATLAB file containing cat and dog images
mat = loadmat('C:/Users/Jimmy Sitompul/Downloads/catdog.mat')

# Print the contents of the MATLAB file
print(mat)
print(len(mat.get("T")))
print([len(a) for a in mat.get("T")])

# Extract the data from the MATLAB file and convert it to
# a Pandas DataFrame
mat1 = pd.DataFrame(mat.get('T'))

# Convert the data to a NumPy array of float32 data type and flatten it
mat2 = np.array(mat1, dtype=np.float32).ravel()

# Print the length of the flattened NumPy array
print(len(mat2))
print(len(mat2)/2)

# Extract the next 99 images (representing dogs) from the flattened array
# and reshape it to a 2D array of length 99 whose elements are sub-arrays
# of length 4096
cat = np.array(mat2[0:405504], dtype=np.float32) #405504*2 = 811008
cat = np.reshape(cat, (99, -1))
for i in range (0,99):
    img = cat[i]
    img = np.reshape(img, (64, 64))
    img = np.rot90(img, k=3)
    img = np.array(img, dtype=np.float32).ravel()
    cat[i]=img
cat_pic = cat[0]

meow = np.reshape(cat_pic,(64,-1))
print(meow.shape)

# Construct the even and odd basis vectors
v_even = (meow + np.fliplr(meow))/2
v_odd = (meow - np.fliplr(meow))/2
```

```python

# Project the image onto each of the basis vectors
c_even = np.sum(meow*v_even)/np.sum(v_even**2)
c_odd = np.sum(meow*v_odd)/np.sum(v_odd**2)



# Symmetrize the image by adding the even component to itself reflected about the vertical axis
# and subtracting the odd component from itself reflected about the vertical axis
sym_cat = c_even*v_even + c_even*np.fliplr(v_even) - c_odd*v_odd - c_odd*np.fliplr(v_odd)

sym_cat = np.array(sym_cat, dtype=np.float32).ravel()
xmin= min(sym_cat)
print(xmin)
xmax = max(sym_cat)
print(xmax)



symmetrized_cat = np.array([])
# Convert the symmetrized image to uint8 format
for i in sym_cat:
    f = lambda i : (85/124)*i+(-170/31)
    val = int(f(i))
    symmetrized_cat = np.append(symmetrized_cat,val)
symmetrized_cat = np.reshape(symmetrized_cat,(64,64)).astype(np.uint8)
print(symmetrized_cat.dtype)

meow= meow.astype(np.uint8)

cv2.imshow('Cat Image', meow)
cv2.imshow('Symmetrized Cat Image', symmetrized_cat)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

9.9 Symmetrized Dog Image

```python
import numpy as np
import cv2  # OpenCV library for image processing
from scipy.io import loadmat  # Library for reading MATLAB files
from scipy.linalg import svd
import pandas as pd  # Library for data manipulation and analysis
import numpy as np
from sklearn.preprocessing import normalize
from sklearn.decomposition import PCA

# Load the MATLAB file containing cat and dog images
mat = loadmat('C:/Users/Jimmy Sitompul/Downloads/catdog.mat')

# Print the contents of the MATLAB file
print(mat)
print(len(mat.get("T")))
```

```python
print([len(a) for a in mat.get("T")])

# Extract the data from the MATLAB file and convert it to
# a Pandas DataFrame
mat1 = pd.DataFrame(mat.get('T'))

# Convert the data to a NumPy array of float32 data type and flatten it
mat2 = np.array(mat1, dtype=np.float32).ravel()

# Print the length of the flattened NumPy array
print(len(mat2))
print(len(mat2)/2)

# Extract the next 99 images (representing dogs) from the flattened array
# and reshape it to a 2D array of length 99 whose elements are sub-arrays
# of length 4096
dog = np.array(mat2[405504:811008], dtype=np.float32) #405504*2 = 811008
dog = np.reshape(dog, (99, -1))
for i in range (0,99):
    img = dog[i]
    img = np.reshape(img, (64, 64))
    img = np.rot90(img, k=3)
    img = np.array(img, dtype=np.float32).ravel()
    dog[i]=img
#avg_dog = np.mean(dog, axis=0)
#print(len(avg_dog))
dog_pic = dog[0]

woof = np.reshape(dog_pic,(64,-1))
print(woof.shape)

# Construct the even and odd basis vectors
v_even = (woof + np.fliplr(woof))/2
v_odd = (woof - np.fliplr(woof))/2

# Project the image onto each of the basis vectors
c_even = np.sum(woof*v_even)/np.sum(v_even**2)
c_odd = np.sum(woof*v_odd)/np.sum(v_odd**2)


# Symmetrize the image by adding the even component to itself reflected about the vertical axis
# and subtracting the odd component from itself reflected about the vertical axis
sym_dog = c_even*v_even + c_even*np.fliplr(v_even) - c_odd*v_odd - c_odd*np.fliplr(v_odd)

sym_dog = np.array(sym_dog, dtype=np.float32).ravel()
xmin= min(sym_dog)
print(xmin)
xmax = max(sym_dog)
print(xmax)

```

```
symmetrized_dog = np.array([])
# Convert the symmetrized image to uint8 format
for i in sym_dog:
    f = lambda i : (255/418)*i+(-255/22)
    val = int(f(i))
    symmetrized_dog = np.append(symmetrized_dog,val)
symmetrized_dog = np.reshape(symmetrized_dog,(64,64)).astype(np.
    uint8)
print(symmetrized_dog.dtype)

woof= woof.astype(np.uint8)

cv2.imshow('Dog Image', woof)
cv2.imshow('Symmetrized Dog Image', symmetrized_dog)
cv2.waitKey(0)
cv2.destroyAllWindows()
```